

Analisi Statistica: Regressione Logistica & Random Forest

Un confronto rigoroso tra due approcci di modellazione, con deployment in tempo reale tramite un'applicazione Flask dockerizzata per l'analisi di dataset.

MACHINE LEARNING

FLASK

SCIKIT-LEARN

A cura di : Emanuele Brivio e Federico Combi

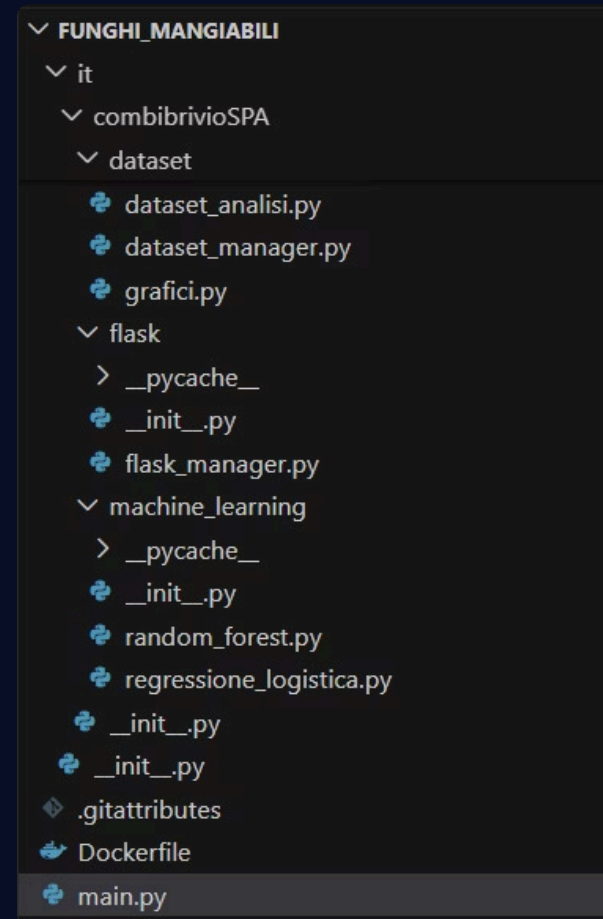
Struttura del progetto:

Sono stati adottati tre package: dataset, flask e machine_learning.

Il package "dataset" contiene le classi e le funzioni necessarie per analizzare e gestire il dataset fornito.

Il package "flask" contiene tutte le classi e funzioni necessarie per realizzare l'interfaccia web.

Il package "machine_learning" consente di splittare i dati in training e test, e addestrare il modello di machine learning scelto.



Il Dataset

Si compone di 8124 osservazioni ciascuno dei quali rappresenta un fungo per cui sono osservate 22 variabili categoriche.

Siamo interessati a stimare se il fungo risulta essere avvelenato oppure no, si tratta quindi di un problema di classificazione binaria.

Variabile Target

Binaria: $Y \in \{0, 1\}$. Rappresenta l'evento da prevedere.

Feature X

Variabili esplicative categoriche, analizzate e trattate per inserirle nel modello.

Obiettivo

Stimare $\Pr(Y=1 | X)$ e assegnare la classe più probabile.

Preprocessing

Per ogni variabile singolarmente si è proceduto ad analizzare:

- il numero di valori nulli.
- la correlazione con le altre variabili categoriche, mediante l'indice di V di Cramér.
- gestire eventuali valori anomali.
- valutare graficamente la distribuzione nelle possibili classi.

Feature engineering:

- rimuovere eventuali duplicati.
- trattare i valori "?" presenti nella variabile "stalk-root" come Nan.
- attribuire ai 2480 valori Nan in "stalk-root", la moda, essendo questa dominante rispetto alle altre classi.
- rimuovere dai dati la colonna "veil-type" in quanto assume un unico valore.

La Regressione Logistica

Stima la probabilità che un'osservazione appartenga a una determinata classe in funzione di un insieme di variabili esplicative, può essere usato quindi anche in presenza di più classi.

Formula chiave

$$\log(p / (1-p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k$$

in termini di odds:

$$p/(1-p) = \exp(\beta_0 + \beta_1 X_1 + \dots + \beta_k X_k)$$

Interpretabilità dei coefficienti.

L'esponenziale del coefficiente, detto **Odds Ratio**, consente di valutare l'impatto della variabile in termine di odds sull'evento.

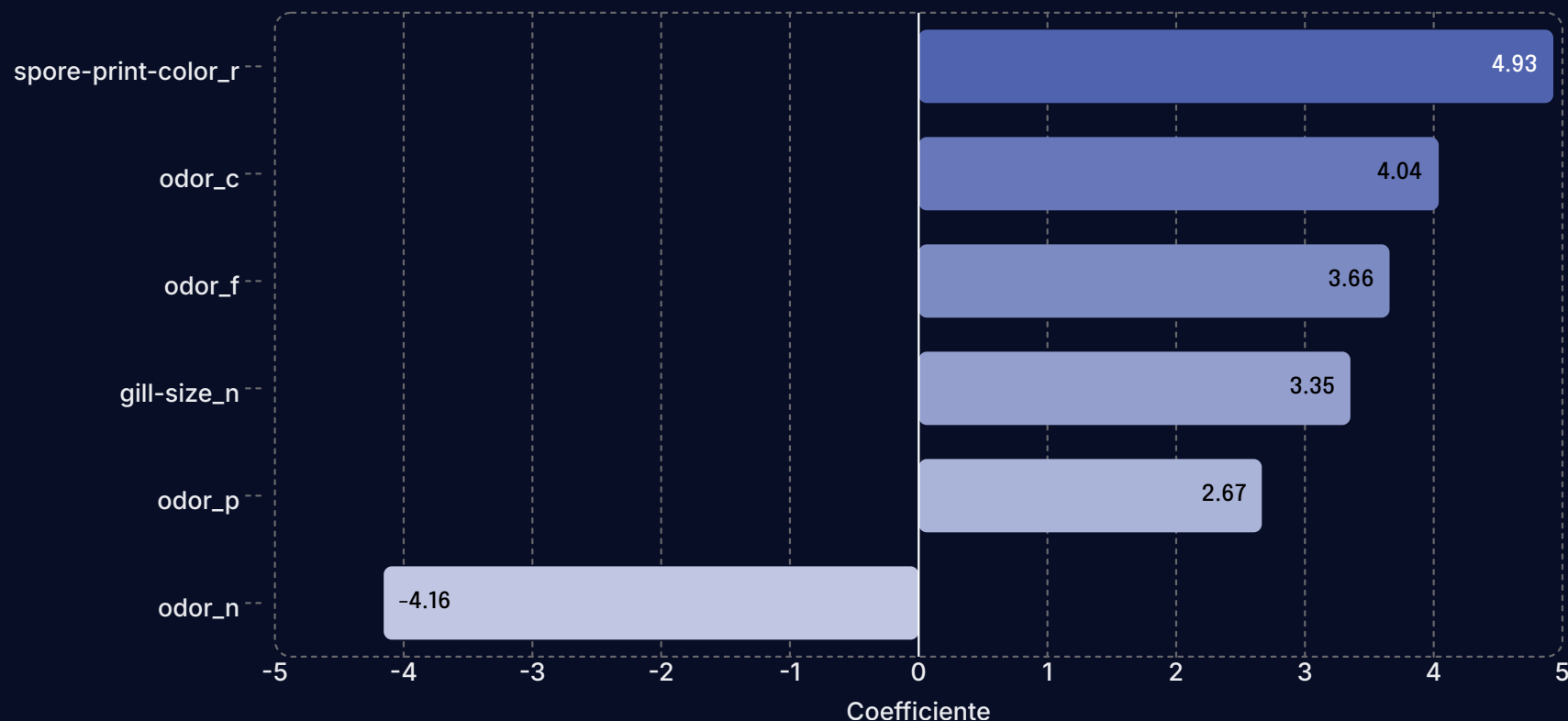
Stima dei parametri

avviene sulla base del principio di **Massima Verosimiglianza**.

Interpretazione Modello - Coefficienti Logistici

Questo grafico mostra i coefficienti del modello di regressione logistica. Ogni coefficiente rappresenta la variazione dei log-odds se viene osservata la specifica classe.

Variabile



Alberi di Classificazione e Regressione

I decision tree sono modelli gerarchici che suddividono ricorsivamente lo spazio delle variabili tramite split binari, costruendo una struttura ad albero in cui ogni nodo rappresenta una regola decisionale.

Alberi di classificazione

Predicono una classe discreta, come sì/no. Gli split vengono scelti per ridurre l'impurità del nodo, tipicamente tramite **Gini impurity** o **entropia**. Nei nodi foglia si assegna l'etichetta di classe più probabile.

Alberi di regressione

Predicono un valore continuo, come prezzo o temperatura. Gli split vengono scelti per massimizzare la **riduzione della varianza** o dell'errore quadratico. Nei nodi foglia si restituisce il **valore medio** delle osservazioni del training contenute.

Interpretabilità

Le regole sono leggibili e facilmente spiegabili.

Relazioni non lineari

Possono catturare pattern complessi nei dati.

Nessuno scaling

Non richiedono la standardizzazione delle feature.

Interazioni

Gestiscono bene combinazioni tra variabili diverse.

Random Forest

Il dataset originale viene ricampionato in più volte, generando più campioni, per ciascuno viene addestrato un albero e le predizioni finali sono una calcolati usando tutte i risultati dei vari campioni.

Come Funziona

- Bootstrap sampling: crea più sottoinsiemi casuali del training set, campionando con reinserimento.
- Random feature selection: a ogni campione si considera solo un sottoinsieme casuale delle feature.
- Train multiple decision trees: ogni albero viene addestrato su un bootstrap sample diverso.
- Aggregation: per la classificazione si usa il voto di maggioranza; per la regressione si fa la media.

Vantaggi:

Gestisce relazioni non lineari

Cattura pattern complessi che un singolo albero può mancare.

Riduce l'overfitting

È in genere più robusto di un albero decisionale isolato.

Feature importance

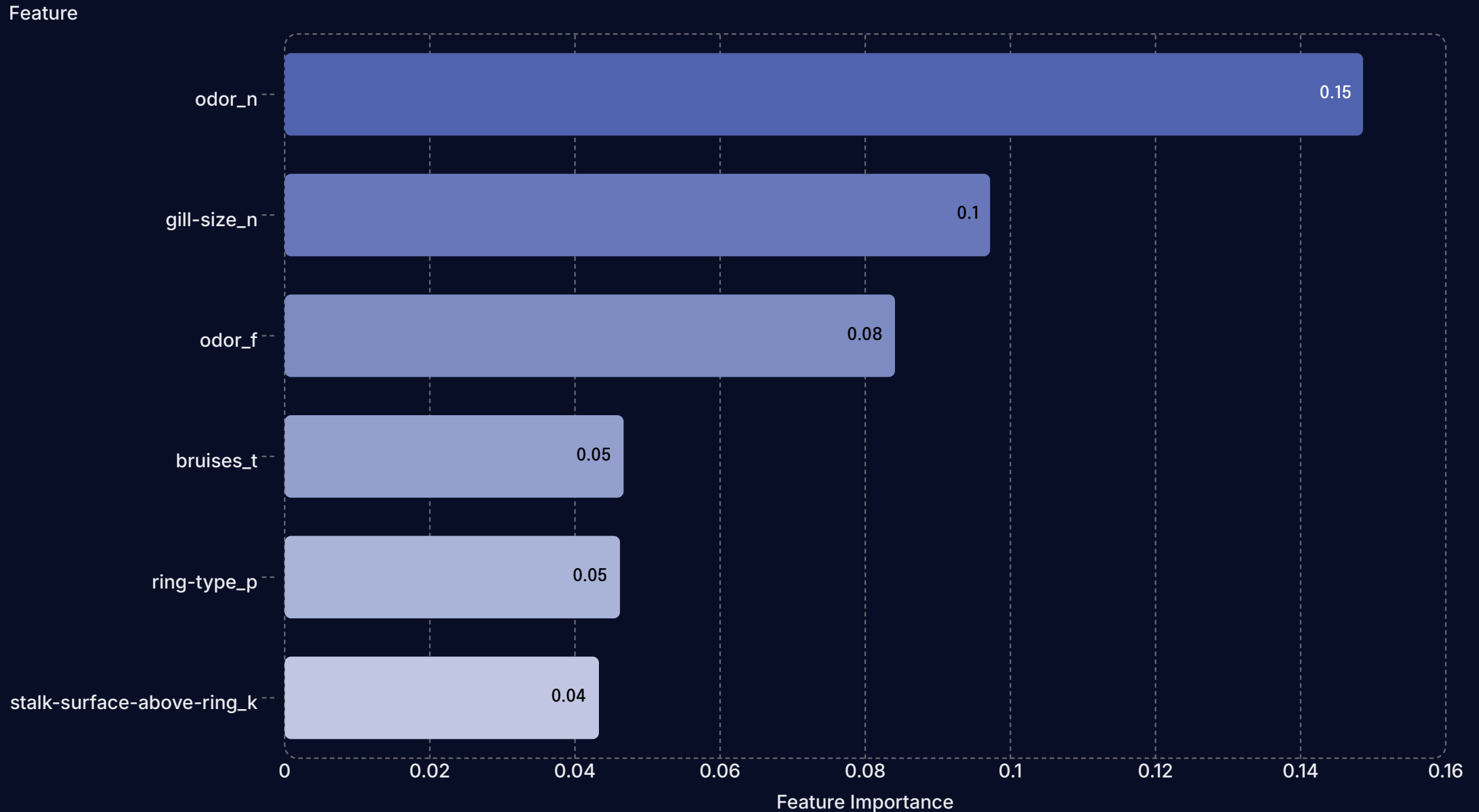
Fornisce una ranking utile dell'importanza delle variabili.

Tipi di feature misti

Può lavorare bene con feature numeriche e categoriche.

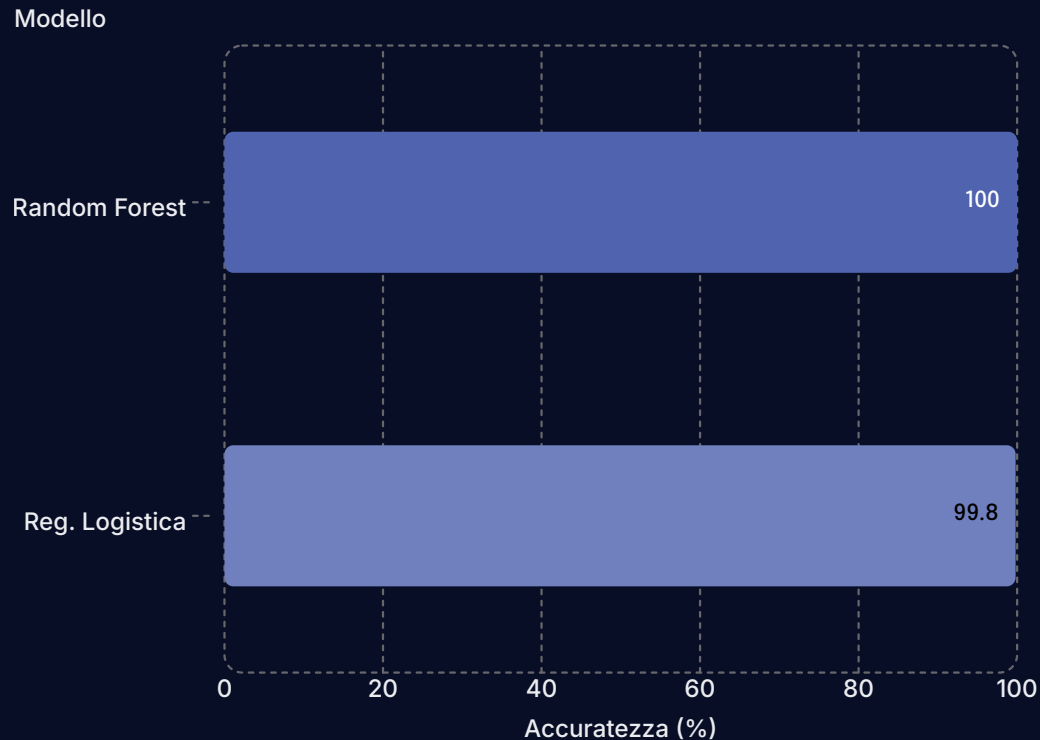
Feature Importance - Random Forest

Questo grafico mostra l'importanza relativa di ciascuna feature nel modello Random Forest, basata su quanto ogni variabile contribuisce a ridurre l'impurità attraverso tutti gli alberi.



Il Confronto:

Valutiamo i due modelli sul test set per identificare il migliore in termini di prestazioni predittive.



Risultati sul dataset

La Random Forest supera la Regressione Logistica di circa **0.2 punti percentuali**, grazie alla capacità di catturare relazioni non lineari tra le variabili.

Questi risultati ci permettono di affermare che entrambi i metodi risultano efficaci nella predizione, e per questo motivo la regressione logistica risulta molto migliore visto il suo ridotto costo computazionale rispetto all'altro.

Flask e Docker

Flask:

- Creato una interfaccia web che permette di analizzare il dataset e creare i due metodi di predizione dei dati.
- Permette dando un oggetto coerente al dataset di ottenere una previsione.

Docker:

- Containerizzato il nostro programma utilizzando Docker Desktop per prepararlo alla distribuzione.
- Viene utilizzato un container rispetto ad una VM per sfruttare al massimo l'hardware a disposizione e consentire una scalabilità maggiore.

Creazione container

Utilizziamo un Dockerfile per creare l'immagine del nostro container.

Dockerfile utilizza "requirements.txt" per importare tutte le librerie utilizzate, questo file viene creato in automatico mediante comando da terminare: `pip freeze > requirements.txt`

```
 Dockerfile
1  FROM python:3.12
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install --no-cache-dir -r requirements.txt
5  COPY . .
6  EXPOSE 5000
7  CMD ["python", "-m", "main"]
```

Sviluppi Futuri:

- permettere all'utente di caricare un suo CSV.
- utilizzare ulteriori metriche per la valutazione dei modelli es. precision, recall.
- ripetere lo split dei dati N volte per ottenere un intervallo di confidenza per l'accuracy. (cambiando il random state durante lo split)
- permettere la visualizzazione del container in rete pubblica.
- rendere più generali delle funzioni per permettere il loro utilizzo in altri contesti es. `clean_data` in `dataset_analisi`.